

src/core/events.rs

```
use crate::core::commands::AppCommand;
use crate::core::models::{
    MarketSnapshot,
    TradeModel,
    TrackedOrder,
};

#[derive(Debug, Clone)]
pub enum AppEvent {
    Market(MarketEvent),
    Order(OrderEvent),
    Trade(TradeEvent),
    Ui(UiEvent),
    System(SystemEvent),
}

#[derive(Debug, Clone)]
pub enum MarketEvent {
    SnapshotUpdated(MarketSnapshot),
    MarketDisconnected(String),
    MarketConnected(String),
}

#[derive(Debug, Clone)]
pub enum OrderEvent {
    OrderUpdated(TrackedOrder),
    OrderRemoved(String),
    OrderRejected {
        order_id: String,
        reason: String,
    },
}

#[derive(Debug, Clone)]
pub enum TradeEvent {
    TradeUpdated(TradeModel),
}

#[derive(Debug, Clone)]
pub enum UiEvent {
    SubmitCommand(AppCommand),

    SelectMarket {
```

```
        market_id: String,
    },
    Refresh,
}

#[derive(Debug, Clone)]
pub enum SystemEvent {
    Shutdown,
    Error(String),
}
```

src/core/state.rs

```
use std::sync::Arc;

use dashmap::DashMap;

use crate::core::models::{
    MarketSnapshot,
    TradeModel,
    TrackedOrder,
};

pub type SharedState = Arc<AppState>;

pub struct AppState {
    pub markets: DashMap<String, MarketSnapshot>,
    pub orders: DashMap<String, TrackedOrder>,
    pub trades: DashMap<String, TradeModel>,
}

impl AppState {
    pub fn new() -> Self {
        Self {
            markets: DashMap::new(),
            orders: DashMap::new(),
            trades: DashMap::new(),
        }
    }
}
```

src/core/reducer.rs

```
use crate::core::events::{
    AppEvent,
    MarketEvent,
    OrderEvent,
    TradeEvent,
};

use crate::core::state::SharedState;

pub fn reduce(
    state: &SharedState,
    event: AppEvent,
) {
    match event {
        AppEvent::Market(event) => {
            apply_market(state, event);
        }

        AppEvent::Order(event) => {
            apply_order(state, event);
        }

        AppEvent::Trade(event) => {
            apply_trade(state, event);
        }

        AppEvent::Ui(_) => {}

        AppEvent::System(_) => {}
    }
}

fn apply_market(
    state: &SharedState,
    event: MarketEvent,
) {
    match event {
        MarketEvent::SnapshotUpdated(snapshot) => {
            state
                .markets
                .insert(snapshot.market_id.clone(), snapshot);
        }

        MarketEvent::MarketDisconnected(_) => {}
    }
}
```

```

        MarketEvent::MarketConnected(_) => {}
    }
}

fn apply_order(
    state: &SharedState,
    event: OrderEvent,
) {
    match event {
        OrderEvent::OrderUpdated(order) => {
            state
                .orders
                .insert(order.order_id.clone(), order);
        }

        OrderEvent::OrderRemoved(order_id) => {
            state.orders.remove(&order_id);
        }

        OrderEvent::OrderRejected { .. } => {}
    }
}

fn apply_trade(
    state: &SharedState,
    event: TradeEvent,
) {
    match event {
        TradeEvent::TradeUpdated(trade) => {
            state
                .trades
                .insert(trade.trade_id.clone(), trade);
        }
    }
}

```

src/core/models/market.rs

```

#[derive(Debug, Clone)]
pub struct MarketSnapshot {
    pub market_id: String,

    pub bid: f64,

```

```
pub ask: f64,  
pub spread: f64,  
  
pub last_price: f64,  
  
pub volume: f64,  
  
pub updated_at_ms: u64,  
}
```

src/core/models/order.rs

```
#[derive(Debug, Clone)]  
pub struct TrackedOrder {  
    pub order_id: String,  
  
    pub market_id: String,  
  
    pub side: String,  
  
    pub price: f64,  
  
    pub quantity: f64,  
  
    pub filled_quantity: f64,  
  
    pub status: OrderStatus,  
  
    pub created_at_ms: u64,  
  
    pub updated_at_ms: u64,  
}  
  
#[derive(Debug, Clone)]  
pub enum OrderStatus {  
    Pending,  
    Open,  
    PartiallyFilled,  
    Filled,  
    Cancelled,  
    Rejected,  
}
```

src/core/models/trade.rs

```
#[derive(Debug, Clone)]
pub struct TradeModel {
    pub trade_id: String,

    pub order_id: String,

    pub market_id: String,

    pub side: String,

    pub price: f64,

    pub quantity: f64,

    pub timestamp_ms: u64,
}
```

src/worker/reducer_task.rs

```
use tokio::sync::mpsc::Receiver;

use crate::core::events::AppEvent;
use crate::core::reducer::reduce;
use crate::core::state::SharedState;

pub async fn start_reducer_task(
    state: SharedState,
    mut rx: Receiver<AppEvent>,
) {
    while let Some(event) = rx.recv().await {
        reduce(&state, event);
    }
}
```

src/worker/batching.rs

```
use crate::core::events::AppEvent;

pub fn drain_batch<T>(<
    rx: &mut tokio::sync::mpsc::Receiver<AppEvent>,
    max_batch: usize,
    output: &mut Vec<AppEvent>,
) {
    while let Ok(event) = rx.try_recv() {
        output.push(event);

        if output.len() >= max_batch {
            break;
        }
    }
}
```

src/infrastructure/repaint_scheduler.rs

```
use std::sync::atomic::{
    AtomicBool,
    AtomicU64,
    Ordering,
};

use std::time::{
    Duration,
    SystemTime,
    UNIX_EPOCH,
};

pub struct RepaintScheduler {
    repaint_pending: AtomicBool,
    last_frame_ms: AtomicU64,
    min_frame_interval_ms: u64,
}

impl RepaintScheduler {
    pub fn new(min_frame_interval_ms: u64) -> Self {
        Self {
            repaint_pending: AtomicBool::new(false),

```

```

        last_frame_ms: AtomicU64::new(0),
        min_frame_interval_ms,
    }
}

pub fn should_repaint(&self) -> bool {
    let now = current_time_ms();

    let last = self.last_frame_ms.load(Ordering::Relaxed);

    if now.saturating_sub(last) >= self.min_frame_interval_ms {
        self.last_frame_ms
            .store(now, Ordering::Relaxed);

        self.repaint_pending
            .store(false, Ordering::Relaxed);

        true
    } else {
        false
    }
}

pub fn request_repaint(&self) {
    self.repaint_pending
        .store(true, Ordering::Relaxed);
}

pub fn repaint_pending(&self) -> bool {
    self.repaint_pending
        .load(Ordering::Relaxed)
}

}

fn current_time_ms() -> u64 {
    SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap_or(Duration::ZERO)
        .as_millis() as u64
}

```


src/app/snapshot/snapshot_cache.rs

```
use crate::core::models::{
    MarketSnapshot,
    TradeModel,
    TrackedOrder,
};

use crate::core::state::SharedState;

pub struct UiSnapshot {
    pub markets: Vec<MarketSnapshot>,
    pub orders: Vec<TrackedOrder>,
    pub trades: Vec<TradeModel>,
}

impl UiSnapshot {
    pub fn build(
        state: &SharedState,
    ) -> Self {
        Self {
            markets: state
                .markets
                .iter()
                .map(|v| v.clone())
                .collect(),

            orders: state
                .orders
                .iter()
                .map(|v| v.clone())
                .collect(),

            trades: state
                .trades
                .iter()
                .map(|v| v.clone())
                .collect(),
        }
    }
}
```